Outlier Detection and Treatment

Introduction

Outliers are data points that significantly deviate from other observations in a dataset. They can indicate variability in your measurement, experimental errors, or novel events. Detecting and treating outliers is vital because they can skew analyses, mislead statistical results, and degrade model performance.

Learning Objectives

By the end of this lesson, learners will be able to:

1. Define and identify outliers in a dataset.
2. Understand different methods for outlier detection.
3. Apply Z-score and IQR-based techniques to detect and treat outliers.
4. Decide when to remove, cap, or retain outliers based on context.
5. Implement these techniques using Python and Pandas.

Concept Explanation

---

1. What Are Outliers?

- A value that lies far outside the range of the majority of the data.
- Could be caused by:
    - Data entry error (e.g., 200 instead of 20).
    - A genuine rare event (e.g., Bill Gates in an income dataset).

---

2. Outlier Detection Methods

A. Visual Methods

- **Box Plot**: Shows outliers as points beyond whiskers.
- **Scatter Plot**: Identifies spread and anomalies in bivariate data.
- **Histogram**: Helps spot skewed distributions and extreme values.

B. Statistical Methods

i. Z-score Method

- Measures how far a value is from the mean in units of standard deviation.
- If `|Z| > 3`, the data point is typically considered an outlier.

ii. IQR (Interquartile Range) Method

- Based on spread between 25th (Q1) and 75th (Q3) percentiles:
  $IQR=Q3−Q1\text{IQR} = Q3 - Q1IQR=Q3−Q1$
- Outliers fall outside:
  $[Q1−1.5×IQR, Q3+1.5×IQR][Q1 - 1.5 \times IQR,\ Q3 + 1.5 \times IQR][Q1−1.5×IQR, Q3+1.5×IQR]$

---

3. Outlier Treatment Methods

A. Z-score Filtering

python

```
import pandas as pd
import numpy as np
from scipy import stats

# Sample Data
df = pd.DataFrame({'value': [10, 12, 14, 15, 18, 19, 120]})

# Calculate Z-scores
```

```
z_scores = np.abs(stats.zscore(df['value']))

# Filter out values with Z-score > 3
df_z_filtered = df[z_scores < 3]
print(df_z_filtered)
```

B. IQR-Based Filtering

python

```
# Calculate Q1 and Q3
Q1 = df['value'].quantile(0.25)
Q3 = df['value'].quantile(0.75)
IQR = Q3 - Q1

# Define bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter out outliers
df_iqr_filtered = df[(df['value'] >= lower_bound) & (df['value'] <=
upper_bound)]
print(df_iqr_filtered)
```

C. Capping (Winsorization)

Instead of removing outliers, replace them with boundary values.

python

```
# Cap values using IQR method
df_capped = df.copy()
df_capped['value'] = np.where(df['value'] > upper_bound, upper_bound,
                      np.where(df['value'] < lower_bound,
lower_bound, df['value']))
print(df_capped)
```

Real-World Applications

- **Finance**: Unusual transactions might indicate fraud.
- **Healthcare**: Sudden spike in vitals might signal emergency.
- **E-commerce**: Very high basket sizes may skew average revenue.

Student-Friendly Analogy

Imagine class test scores:

- If most students score between 50–80, but one student scores 0 or 100, they’re outliers.
- If you're analyzing class performance, that one student could distort the average.
- You could:
    - Ignore them (remove).
    - Replace their score with the highest/lowest within range (cap).
    - Keep them if you’re specifically studying anomalies.

Common Mistakes & Misunderstandings

- **Blindly removing outliers**: Might lose critical insights.
- **Applying Z-score to non-normal data**: Can mislead detection.
- **Using IQR on very small datasets**: May result in no outliers or false positives.
- **Not visualizing**: Always plot before deciding to remove or treat outliers.

Interactive Exercises

Exercise 1: Outlier Detection Practice

Load the "tips" dataset from seaborn and:

1. Plot a boxplot for `total_bill`.
2. Identify outliers using Z-score.
3. Detect outliers using IQR method.
4. Cap the extreme values in `total_bill`.

python

```
import seaborn as sns
df = sns.load_dataset("tips")

# Boxplot
sns.boxplot(x=df['total_bill'])

# Z-score method
z = np.abs(stats.zscore(df['total_bill']))
print(df[z > 3])

# IQR method
q1 = df['total_bill'].quantile(0.25)
q3 = df['total_bill'].quantile(0.75)
iqr = q3 - q1
outliers = df[(df['total_bill'] < q1 - 1.5*iqr) | (df['total_bill'] >
q3 + 1.5*iqr)]
print(outliers)
```

Recommended Resources

- Book: *An Introduction to Statistical Learning* by Gareth James et al.
- Blog: Analytics Vidhya – Outlier Detection Techniques

- Video: Krish Naik – “Outlier Detection in Python”
- Dataset: Kaggle – House Prices(for practice)

Wrap-Up / Recap

- Outliers are unusual data points that can skew results.
- Use visual (boxplots) and statistical (Z-score, IQR) methods to detect them.
- Treat them by filtering, removing, or capping values.
- Always analyze impact before and after treatment.

Quiz / Reflection Questions

1. What are the advantages of capping over removal of outliers?
2. When is it more appropriate to use IQR rather than Z-score?
3. How can outliers affect machine learning models?
4. Can an outlier ever be valuable to keep in your dataset? Why?